Welcome to the Python Course!

This document compiles the advanced machine learning and AI integration topics covered during the fourth and final week of the course. It explains basic dataset modeling, API communication with Groq models, and details our final AI-powered clinical recommendation project.

Table of Contents / Topics Covered:

1. Machine Learning Basics & Model Training

On Day 18, we introduced Machine Learning (ML) basics using the **`scikit-learn`** (sklearn) and **`pandas`** libraries. We walked through the complete pipeline of building a predictive model: loading data, splitting datasets into features and target variables, performing a training and testing split, and training/testing both a **Linear Regression** and a **Random Forest Regressor** model to predict house prices.

1. Data Collection & Inspection

In this phase, we load a house price dataset containing attributes like area (in sqft), number of bedrooms, and the age of the house to predict its price in lakhs. We leverage the **`pandas`** library to load and inspect the dataset.

Python Code (ML.ipynb)

```
import pandas as pd
# Load dataset
dataset = pd.read_csv(&#x27;data.csv')
# Inspect the first few rows
dataset.head()
```

Dataset Structure (data.csv):

| Column Name | Description | | :--- | :--- | | **`Area_sqft`** | The size of the house in square feet (Feature) | | **`Bedrooms`** | The number of bedrooms in the house (Feature) | | **`Age_years`** | The age of the house in years (Feature) | | **`Price_lakhs`** | The price of the house in lakhs (Target Variable) |

■ 2. Feature & Target Splitting

To train a machine learning model, we must separate our dataset into:

- **Features (`X`)**: The input variables used to make predictions.
- **Target (`y`)**: The output variable we want the model to predict.

We use integer-location based indexing (**`.iloc`**) from pandas to slice the DataFrame.

Python Code (ML.ipynb)

```
# Extract all columns except the last one as features
X = dataset.iloc[:, :-1]
# Extract the last column as the target variable
y = dataset.iloc[:, -1]
```

---

3. Training & Testing Split

To evaluate our model's performance on unseen data, we divide our features and target into training sets (for model learning) and testing sets (for evaluation). We use **train_test_split** from **`sklearn.model_selection`**.

Python Code (ML.ipynb)

```
from sklearn.model_selection import train_test_split
# Split into 70% training and 30% testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

- **`test_size = 0.3`** reserves 30% of the dataset for testing.
- **`random_state = 42`** ensures reproducible splits across different runs.

---

4. Linear Regression

LinearRegression establishes a linear relationship between the input features and the continuous target variable.

Python Code (ML.ipynb)

```
from sklearn.linear_model import LinearRegression
# Instantiate the model
model = LinearRegression()
# Train the model using the training sets
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)
print(y_pred)
```

---

5. Random Forest Regressor

RandomForestRegressor is an ensemble learning method that constructs multiple decision trees during training and outputs the average prediction of the individual trees for regression tasks.

Python Code (ML.ipynb)

```
from sklearn.ensemble import RandomForestRegressor
# Instantiate the model
model = RandomForestRegressor()
# Train the model using the training sets
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)
print(y_pred)
```

2. Large Language Models (LLMs) & API Integration with Groq

On Day 19, we introduced Large Language Models (LLMs) and learned how to integrate them into Python applications using APIs. We covered the fundamentals of APIs, the purpose of API keys, creating an API key from the Groq console, and building a simple query-answer system using the `groq` library in Python.

1. Key Concepts

• **LLM (Large Language Model)**: Deep learning algorithms that can recognize, summarize, translate, predict, and generate text and other content based on knowledge gained from massive datasets.

• **API (Application Programming Interface)**: A set of protocols that allows different software applications to communicate with each other. In our case, we send inputs (prompts) to the Groq API and receive outputs (responses) generated by the model.

• **API Key**: A unique code passed to an API to identify the calling program or user. It acts as an access token to authenticate requests.

2. Creating an API Key from Groq

1. Visit the **Groq Console**.
2. Create an account or sign in.
3. In the sidebar, click on **API Keys**.
4. Click **Create API Key**, provide a name, and copy the generated key. *Keep this key private!*

■ 3. Creating the Client & Making Queries

Using the `groq` library in Python, we initialize the client with our API key and request responses from an LLM.

Client Initialization (main.ipynb)

```
from groq import Groq
# Store your API key (Replace with your actual key)
GROQ_API_KEY = &#x27;YOUR_API_KEY'
# Create the Groq client
client = Groq(
  api_key=GROQ_API_KEY
)
```

Query-Answer System (main.ipynb)

```
# Get user prompt
prompt = input("Enter your prompt: ")
# Generate response from Llama-3.3-70b-versatile
response = client.chat.completions.create(
  model = "llama-3.3-70b-versatile",
  messages = [
    {
      &#x27;role': 'user',
      &#x27;content': prompt
    }
  ]
)
# Display the output
print(response.choices[0].message.content)
```

3. Final Project - AI-Powered Diabetes Prediction & Inference

On Day 20, we built our final project: an interactive AI-powered diagnostic and recommendation system for diabetes. The application integrates traditional Machine Learning (classification) with generative Large Language Models (LLMs) to predict diabetes probability and deliver personalized medical guidance based on a patient's health metrics.

Project Structure

The project is organized under the **final_project** directory:

- **data.csv**: A dataset containing historical patient data (Age, BMI, Glucose, Blood Pressure, and Diabetes label).
- **model.py**: Logistic Regression classifier trained on patient data to predict the presence of diabetes.
- **guidance.py**: Integrates the Groq API and `llama-3.3-70b-versatile` model to generate custom clinical guidance.
- **main.py**: The main script prompting user inputs, running model inference, and calling the LLM guidance module.

---

1. The Dataset

The model trains on a 5-column dataset capturing essential clinical indicators.

Feature Name	Description
`Age`	Age of the patient in years
`BMI`	Body Mass Index (weight in kg / height in m^2)
`Glucose`	Blood glucose concentration
`BloodPressure`	Diastolic blood pressure (mm Hg)
`Diabetes`	Target label: `0` (No Diabetes), `1` (Diabetes)

---

2. Model Training & Prediction

Using `scikit-learn`, we train a **Logistic Regression** model. We split the data into training and testing sets, fit the model, and expose a prediction interface.

Python Code (model.py)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
# Importing the dataset
dataset = pd.read_csv(&#x27;data.csv')
# Splitting the dataset into Features and Target
X = dataset.iloc[:,:-1]
y = dataset.iloc[:,-1]
# Split the dataset into training and testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialized and train the model
model = LogisticRegression()
model.fit(X_train, y_train)
# Testing / Prediction function
def predict(age, bmi, glucose, bp):
  y_pred = model.predict([[age, bmi, glucose, bp]])
  if y_pred[0] == 0:
    return &#x27;No Diabetes'
  return &#x27;Diabetes'
```

3. AI-Powered Lifestyle Inference & Guidance

We utilize the Groq API and a state-of-the-art LLM to generate custom lifestyle suggestions for the patient. By feeding the user's vitals and the ML model's prediction as context, the LLM provides personalized, natural language recommendations.

Python Code (guidance.py)

```
from groq import Groq
GROQ_API_KEY = &#x27;YOUR_API_KEY'
client = Groq(api_key=GROQ_API_KEY)
def getGuidance(age, bmi, glucose, bp, prediction):
  prompt = f"""
    You are an expert in diabetes diagnosis. The following data includes age, BMI, glucose, blood pressure a
    Data: Age: {age}, BMI: {bmi}, Glucose: {glucose}, Blood Pressure: {bp}, Prediction: {prediction}
    """
  response = client.chat.completions.create(
    model = &#x27;llama-3.3-70b-versatile',
    messages = [
      {
        &#x27;role': 'user',
        &#x27;content': prompt
      }
    ]
  )
  return response.choices[0].message.content
```

4. Putting It All Together

The application entry point accepts user parameters via command-line prompts, runs the machine learning classification, triggers LLM inference, and prints detailed guidance.

Python Code (main.py)

```
from model import predict
from guidance import getGuidance
# Get input from patient/clinician
age = int(input("Enter your age: "))
bmi = float(input("Enter your BMI: "))
glucose = float(input("Enter your glucose: "))
bp = int(input("Enter your BP: "))
# Predict using Logistic Regression Model
prediction = predict(age, bmi, glucose, bp)
# Fetch personalized guidance from Llama LLM via Groq
print(getGuidance(age, bmi, glucose, bp, prediction))
```

Practice Questions

1. Machine Learning Basics

Question 1 — Model Evaluation Metric (MSE) ■

In Week 4, we trained a Linear Regression model. To understand how well our model fits continuous values, we compute the **Mean Squared Error (MSE)**.

Write a Python script that loads your test variables, runs predictions, and calculates the MSE without using `sklearn.metrics`. The formula for MSE is:

```
# Formula: MSE = (1 / n) * sum((y_actual - y_predicted) ** 2)
y_actual = [30.5, 45.0, 15.2, 50.1, 22.8]
y_predicted = [31.0, 42.5, 16.0, 49.0, 24.0]
```

Requirements:

• Iterate through both lists and calculate the squared difference for each element.

• Sum the squared differences and divide by the total number of samples.

• Verify the output by printing the calculated MSE.

Question 2 — Customer Feedback Classifier ■

Create a simple Logistic Regression classification pipeline using `scikit-learn` to classify text feedback based on simple feature counts (e.g. number of positive/negative words).

Feedback Text	Num_Positive_Words	Num_Negative_Words	Label (1=Pos, 0=Neg)
"Great product, highly recommend!"	2	0	1
"Terrible customer service and bad experience."	0	2	0
"Average quality, not bad but not good."	1	1	0
"Amazing service, I love it."	3	0	1

Requirements:

• Initialize features `X` (positive word count, negative word count) and target labels `y`.

• Train a `LogisticRegression` model.

• Test the model by passing a custom input `[2, 0]` and check if it predicts `1` (Positive).

2. LLM API Integrations

Question 3 — AI Code Helper / Reviewer ■

Build a Python application utilizing the Groq API (`llama-3.3-70b-versatile`) to act as an automated Code Reviewer.

Requirements:

• Prompt the user to enter a Python function they wrote.

• Prepare a system message instructing the model to analyze the code for time complexity, space complexity, and potential bugs, then output optimized code.

• Print the model's review details formatted clearly in the console.

```
system_instruction = """
You are an expert Python compiler and reviewer. Analyze the code provided by the user.
Provide: 1. Code Review (bugs/efficiency), 2. Optimized Version of the function.
"""
```

3. End-to-End Projects

Question 4 — Clinical Record Keeper & AI Diagnosis ■

Enhance the diabetes final project by integrating a database logger. Store patient information in SQLite and fetch recommendations using LLM.

Workflow & Architecture:

• When a patient provides age, BMI, glucose, and blood pressure, insert a row into a SQLite table named `patient_records`.

• Pass the values to the trained Logistic Regression model from `model.py` to get the diabetes prediction.

• Update the database record with the prediction (`No Diabetes` or `Diabetes`).

• Invoke the Groq client to get lifestyle recommendations, and display them to the user.

```
CREATE TABLE IF NOT EXISTS patient_records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    age INTEGER, bmi REAL, glucose REAL, bp INTEGER,
    prediction TEXT, timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```